Urban Mobility Dashboard for NYC Yellow Taxi Trips

Technical report.

Team: **Team Grok**

Members:

- INEZA Elsie Glenna
- GENDANEZA KEZA Sonia
- KAYINABABO Hillary
- INGABIRE Ines

1. Problem Framing and Dataset Analysis

Dataset and context. We used the NYC Taxi and Limousine Commission (TLC) yellow taxi records, supplied as three linked files: a fact table of trips (pickup and dropoff times, distance, rate code, itemised charges, tip, payment type, pickup and dropoff zone IDs), a zone lookup of 265 rows (zone ID to borough, zone name and service zone), and a shapefile with the boundary polygons of the taxi zones. Our trips file covers **January 2019** and holds **7,667,792** raw trips. To keep the dashboard responsive on student laptops, we processed a random sample of **200,000** trips. The goal is to let a user explore when, where, and how New Yorkers ride taxis.

Data challenges. Real records are messy. Of the 200,000 sampled trips, 2,931 (1.47%) broke our rules, and another 2,928 (1.46%) had an invalid passenger count (0, blank, or above six). The most common problem was trips under one minute (1,774), followed by trips over six hours (504) and trips with zero distance (308). Only 17 trips were dated outside the main month. In a 19-row extract we inspected by hand, two trips were $52 flat-rate rides between the same non-airport zone, and four had a total $2.75 higher than their itemised charges. The spatial file also needed work: boundaries were in New York State Plane feet (EPSG:2263), not latitude and longitude; zones 56 and 103 were split into several polygons; zones 57, 104 and 105 had no polygon; and zones 264 and 265 mean "Unknown".

Cleaning rules. A trip is removed and written to `excluded_trips.csv` with its reason if a critical value is missing; its pickup is outside the file's main month; its duration is not positive, under 1 minute, or over 6 hours; its distance is zero, negative, or over 100 miles; its fare is zero, negative, or over $500; its total is not positive; any fee is negative; a zone ID is not in the lookup; its implied speed is over 80 mph; or it duplicates an earlier trip. We excluded

2,931 of 200,000 sampled trips (1.47%). The biggest reason was trips shorter than one minute: 1,774 trips, 61% of all exclusions.

Assumptions.

- A passenger count of 0 or blank is a driver entry omission, so we set it to 1 (these trips are counted, not hidden).
- Blank fee columns mean no charge, so they become $0.
- Cash tips are not recorded in the data, so tip percentage is calculated only for card payments. Otherwise, cash tips would look like they received no tip.
- The thresholds above are judgment calls based on physical limits, because the data has no ground truth.

Derived features.

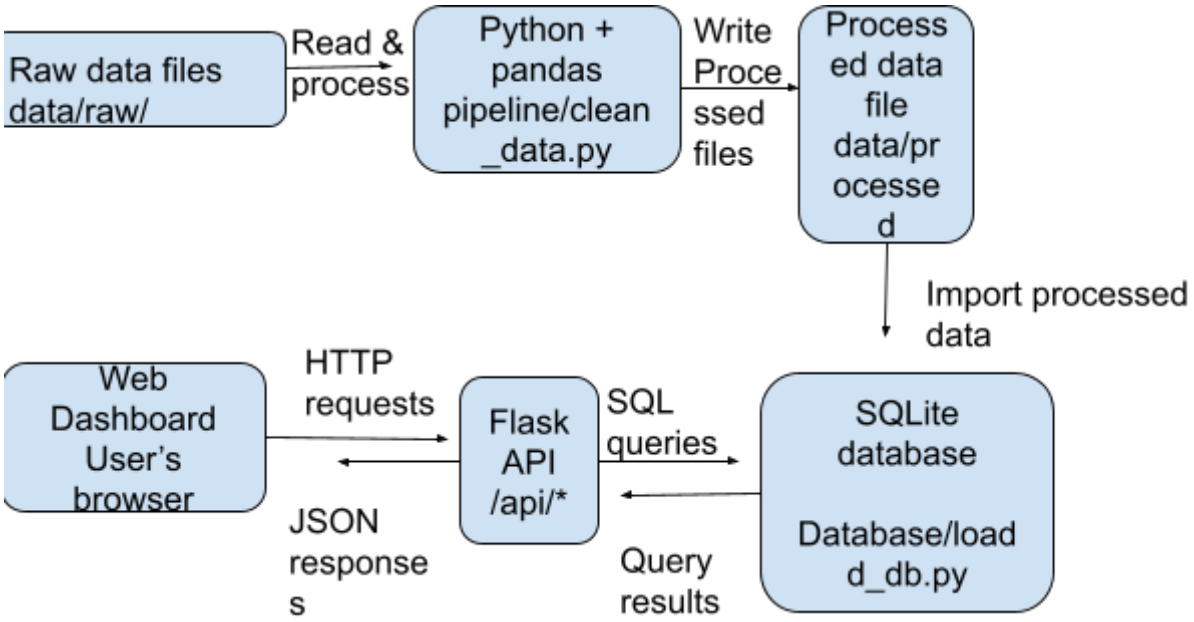
Feature	Calculated from	Why it matters
avg_speed_mph	distance/duration	Shows congestion
fare_per_mile	fare/distance	Shows what a mile of movement costs
tip_pct	tip/fare (card only)	Compares generosity across places and times
is_airport_trip	pickup or dropoff zone's service zone	Separates a distinct kind of trip
is_cross_borough	pickup borough vs dropoff borough	Measures movement between boroughs
pickup_hour, weekday, is_weekend	pickup timestamp	Supports demand patterns

One unexpected observation. In the 19-row extract we first inspected, five trips were dated in the wrong month, all with zero distance, and four had an unexplained $2.75 gap. That fits our design: the pipeline detects each file's dominant month automatically instead of

hard-coding a date range, and every exclusion is logged and shown in the dashboard. The full sample told a different story: fewer than 0.01% of trips (17 of 200,000) were outside the main month. The small extract overstated the problem, reminding us to test assumptions on the full dataset. The real problem was duration: 1,774 trips lasted under one minute, and 504 lasted more than six hours.

2. System Architecture and Design Decisions

Figure 1. System architecture.



The diagram separates the system into two main parts: the batch data pipeline and the live dashboard application (The upper part is the batch data pipeline, while the lower part is the live dashboard application).

1. Batch data pipeline

This prepares the data before the dashboard uses it.

1. Raw data files: The system starts with NYC taxi trip data and supporting zone lookup files, such as CSV, ZIP, and Parquet files.
2. Python + pandas cleaning pipeline: `pipeline/clean_data.py` reads the raw data, cleans and validates records, filters invalid trips, and creates derived features.
3. Processed data files: The pipeline writes the cleaned trips to Parquet, zone information to JSON, and data-quality summaries to CSV.
4. SQLite database: `Database/load_db.py` loads the processed data into SQLite. The database contains the trips fact table, lookup tables, and exclusion summary.

The arrows in this section represent the direction in which data moves during processing and loading.

2. Live dashboard application

This is the part that users interact with.

5. Flask API: The backend exposes endpoints that the dashboard calls. It accesses the SQLite database in read-only mode and uses parameterized queries and an allowlist for sorting.

6. Web dashboard: The frontend uses HTML, CSS, and JavaScript, with Chart.js for visualizations and a custom SVG map. It sends HTTP requests to the API and receives JSON responses to update charts, tables, statistics, and maps.

How the communication is bidirectional

The two-way arrows are important because communication does not stop after a request is sent.

- Dashboard ↔ Flask API: The dashboard sends an HTTP request, such as asking for trips within a selected date range. The API returns a JSON response containing the requested results.
- Flask API ↔ SQLite: The API sends SQL queries to the database, and the database returns the matching rows or aggregated results. The API then formats those results into a response for the dashboard.

The dashboard never accesses SQLite directly. All database communication passes through Flask, which keeps the architecture organized and centralizes data access.

Stack choices: The pipeline uses Python and pandas because they read parquet files and clean data in bulk. The database is SQLite: it needs no setup, it is a single file, and anyone can run it, while still enforcing foreign keys and CHECK constraints. The backend is Flask, in the same language as the pipeline, and opens the database read-only, uses parameterised queries, and only accepts sort columns from a fixed list. The frontend is plain HTML, CSS, and JavaScript, with Chart.js for charts and a hand-drawn SVG map, so no map library is needed. Flask also serves the frontend, so one command starts the whole app.

3. Algorithmic Logic and Data Structures

Problem. The dashboard must show the top K trips, zones, and routes under any combination of filters. Sorting every matching row costs O(n log n) and holds all rows in memory.

Approach. We wrote a binary min-heap by hand (`backend/algorithms.py`), stored in a plain list where the parent of index i is (i - 1) // 2. The heap keeps the K best items seen so far, and its root is the weakest of them. Rows are streamed from the database one at a time, so memory stays at K items. Ties keep the item that came first. We do not use `heapq`, `sorted`, `Counter,` or `sort_values`. Three endpoints use it: longest trips, busiest pickup zones, and busiest routes.

Pseudocode.

```
top_k(stream, K):
    heap = empty min-heap
    for item in stream:
        score = key(item)
        if heap.size < K:
            heap.push(score, item)
        else if score > heap.peek().score:
            heap.pop()              # drop the weakest of the K
            heap.push(score, item)
    result = []
    while heap not empty:           # weakest first, so fill from the back
        result.prepend(heap.pop())
    return result                   # best first
```

Complexity. For n items and K kept: time O(n log K), because each item costs at most one push and one pop on a heap of size K, compared with O(n log n) for sorting everything. Space is O(K). Push and pop are O(log K); peek is O(1).

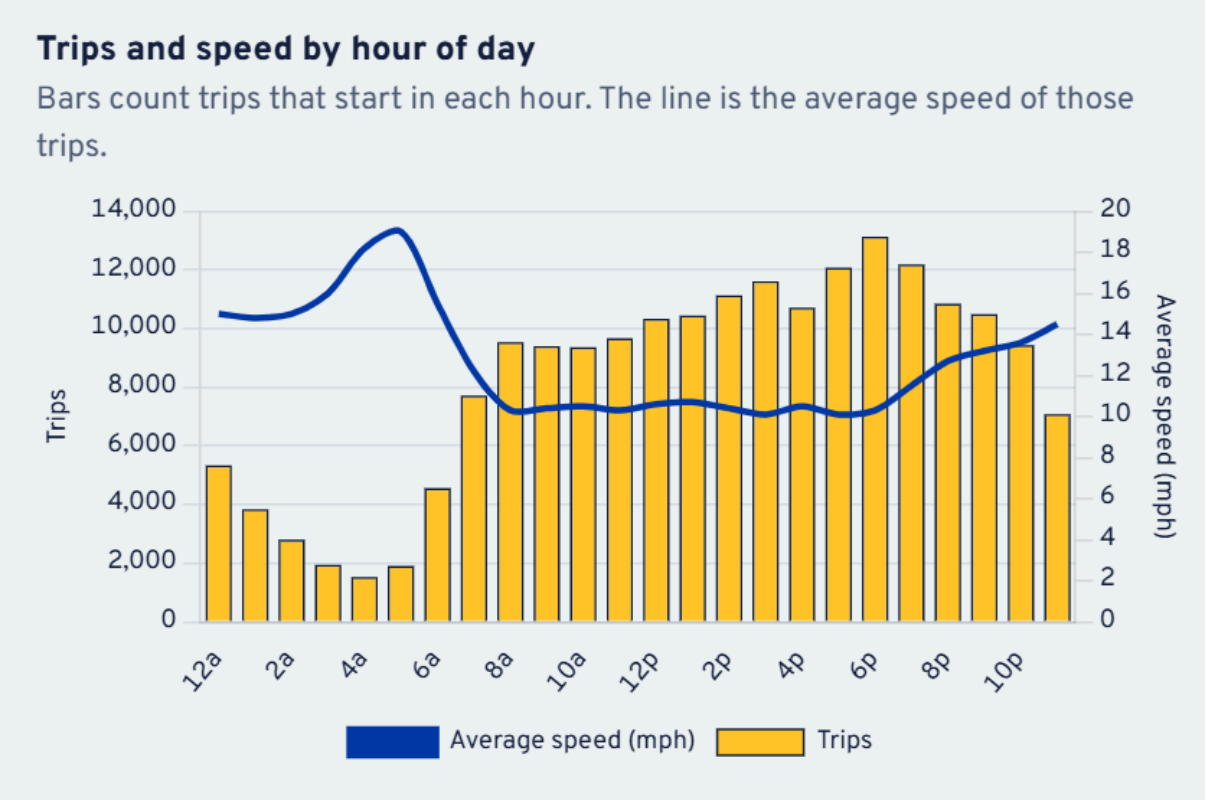
Validation. We compared the heap against brute force on 300 random trials, including ties and missing values, and the API tests compare its results with SQL `ORDER BY ... LIMIT`. 300 random trials x 2 directions passed (including ties and missing values): 1,000,000 numbers, top 10: hand-made heap 0.97s; full sort 1.59s. The speeds are about equal because Python's sort is written in C. The real advantage is memory: the heap holds 10 items, however many trips match, which matters when streaming millions of rows.

4. Insights and Interpretation

All figures come from `database/insight_report.py`, run on our 197,069 cleaned trips (a random sample of January 2019). Speeds are averages of each trip's own speed.

Insight 1. Taxis crawl at about 10 mph all day, not only at rush hour.

- *How derived:* the dashboard chart "Trips and speed by hour" (query 1 in `insight_queries.sql`).

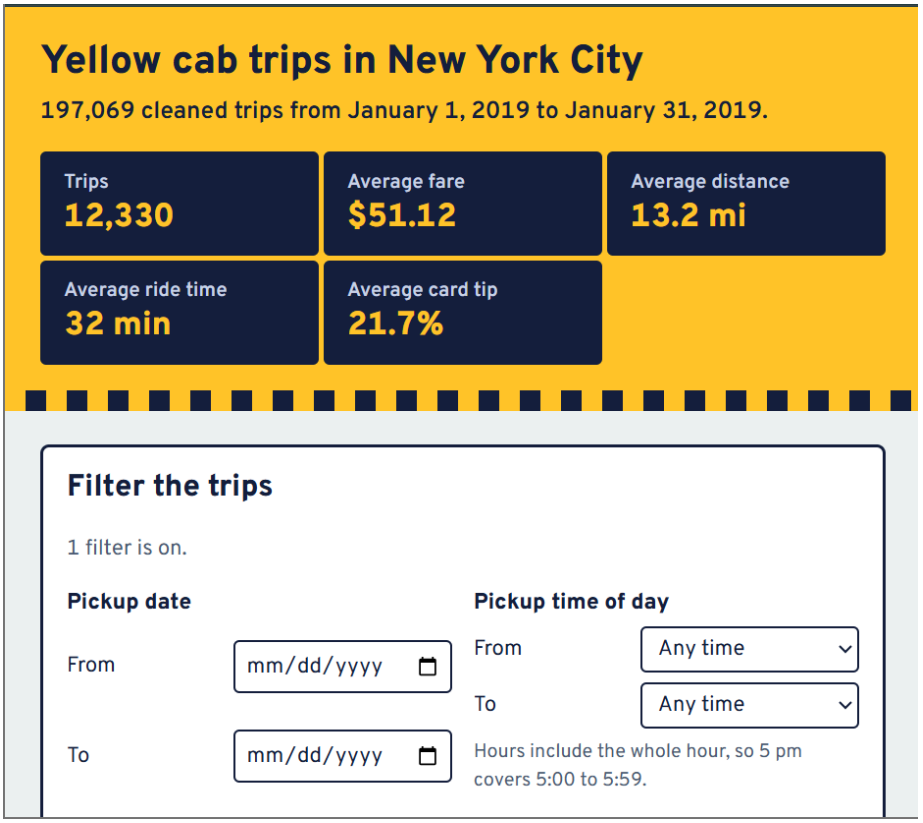


- *Visual:* **the "Trips and speed by hour" chart, no filters on**

- *Interpretation:* Demand peaks at 6 pm (13,129 trips, 6.7% of all trips) and is lowest at 4 am (1,528 trips, 0.8%). Speed does not simply fall at the peak. Every hour from 8 am to 6 pm averages between 10.1 and 10.7 mph, and the slowest hours are 3 pm and 5 pm (10.1 mph). Weekday trips starting 7 to 9 am and 4 to 7 pm average 10.3 mph, against 15.8 mph from midnight to 6 am, a drop of about a third; 5 am is the fastest hour at 19.0 mph. Congestion in Manhattan lasts the whole working day, so a measure aimed only at the two rush-hour peaks would miss most of the slow hours.

Insight 2. Airport trips are 6% of trips but 21% of revenue.

- *How derived:* ticking "Airport trips only" and comparing the headline numbers with no filter (query 2).

- 

- 

- *Visual: Dashboard header numbers with "Airports only" ticked*

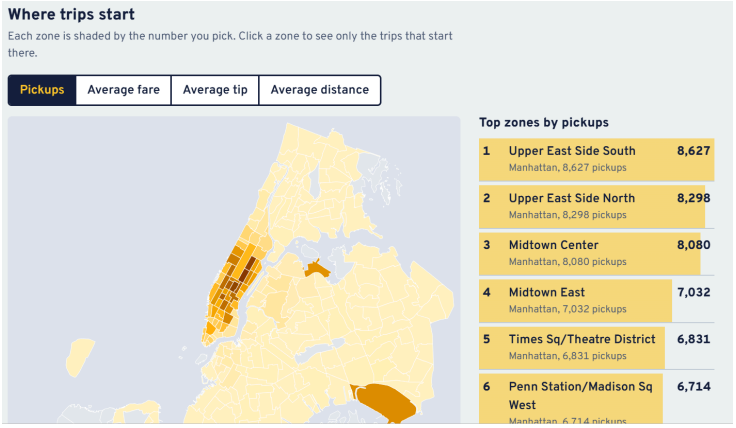
	Airport trips	All other trips
Trips	12,330 (6.3%)	184,739 (93.7%)
Average distance	13.2 mi	2.1 mi
Average ride time	32.0 min	11.7 min

Average total paid	$51.12	$12.98
Share of revenue	20.8%	79.2%
Average card tip	21.7%	21.6%

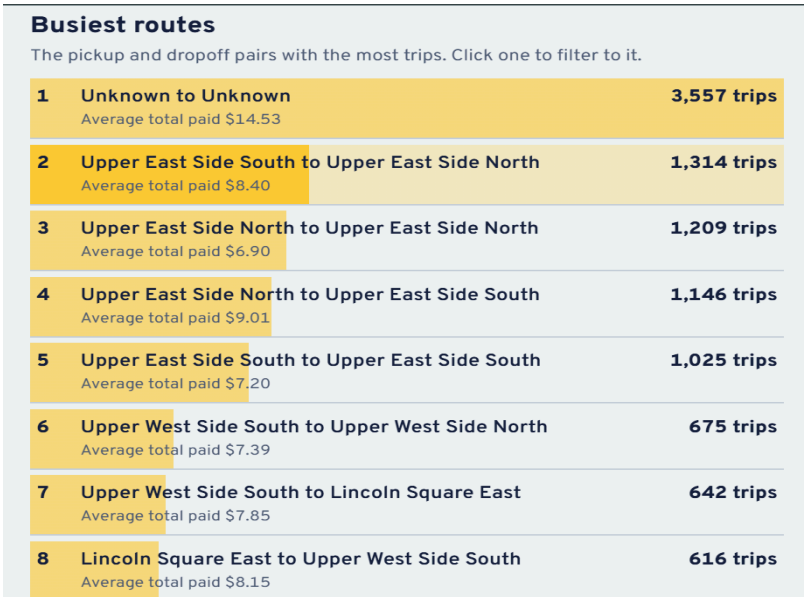
- *Table: Dashboard header numbers with "Airports only" ticked*

- *Interpretation:* An airport trip pays about four times as much as an ordinary trip, so airports bring in 3.3 times more revenue than their share of trips. Trips starting at JFK (4,807) and LaGuardia (4,626) make up nearly all of them. Tips are the same in both groups (21.7% and 21.6%), so airport trips earn more because they are longer, not because passengers pay extra. Trips to and from the airports carry about a fifth of taxi revenue, which makes the roads to them important for the whole taxi system.

Insight 3. Demand is concentrated in Manhattan, and the busiest routes are short local hops.

- *How derived:* the map "Where trips start" shaded by pickups, the ranked zone list, and the "Busiest routes" list, which our hand-written heap produces (query 3).
- 

- *Visual:* **The map shaded by pickups with the "Top zones by pickups" list beside it.**



- *Visual:* **The "Busiest routes" list**

- *Interpretation:* Manhattan has 90.9% of pickups, Queens 5.8% (mostly the two airports), and Brooklyn only 1.1%. The 16 busiest pickup zones, out of 236 with any pickups, produce half of all trips, and the top ten (35.5%) are all in Manhattan, led by Upper East Side South (4.4%). Eight of the ten busiest routes are short hops between neighbouring zones on the Upper East and Upper West Sides, with average fares of $6.90 to $9.01. The single busiest "route" is Unknown to Unknown (3,557 trips, 1.8% of all trips), trips whose zones the TLC could not record, which is a limit of the data. Yellow-taxi activity is concentrated, not city-wide. The data cannot say why the outer boroughs have so few, but other taxi services may cover them.

5. Reflection and Future Work

Technical challenges. The boundary file used a projected coordinate system and had split and missing zones, so we converted it to latitude and longitude and merged the split zones. With no ground truth, we chose cleaning thresholds based on judgment, so we logged every exclusion. The full file is too large for laptops, so we read only the needed columns and used a random sample. Before the real data was available we tested the whole chain on generated trips that copy the real mistakes. Finally, our hand-written heap did not beat Python's built-in sort in raw speed, and we report that honestly. Pushing the large yellow_taxi file failed because it was too large.

For a real product, you'd have to:

- Move to PostgreSQL with PostGIS for concurrent users and spatial queries.
- Pre-compute summary tables so charts load instantly on years of data, and use keyset pagination for the trips table.

- Add several months and a trend view, and join weather and event data to explain demand changes.
- Use the unitemised-charge gap and trip patterns for anomaly detection on suspect meters.
- Add authentication, deployment, automated tests, and accessibility testing.